

Nexa Pulse

Build Roadmap · Real-Time Geospatial Agentic Platform

"Nexa Pulse is the real-time operations command center for a ride-hailing platform — a geospatial AI system that lets marketplace operations managers reason over live driver positions and company policy through natural language."

THE PRODUCT

An operator dashboard with a live map at its center — drivers, riders, demand heat zones, and surge areas updating continuously — paired with a chat panel where an operator asks questions in plain English and receives grounded, cited answers streamed back in real time.

THE USER

A **marketplace operations manager**, not a driver dispatcher. They steer the marketplace through incentives, pricing, and policy — never commanding individual (independent) drivers. The AI advises; the human decides and acts. Human-in-the-loop decision support.

What carries over · what's new

Roughly 80% of the AI/ML stack is identical to the original geospatial project. You are re-skinning a stack you already understand onto a more exciting, real-time product.

Component	Status	What changes
InferEdge (local inference)	Reused	Nothing — already built & working
PostGIS (spatial DB)	Reused	Driver points & demand zones instead of zoning polygons
pgvector (vector DB)	Reused	Operational policies instead of planning PDFs
Hybrid RAG pipeline	Reused	Same chunking, embeddings, spatial+vector query
LangGraph multi-agent	Reused	Same supervisor + agents over mobility data
Embedding fine-tuning	Reused	Mobility vocabulary instead of planning vocabulary
Ragas evaluation	Reused	Evaluates mobility Q&A instead of planning Q&A
Real-time data layer	NEW	Simulated moving drivers + live updates (WebSockets)
React frontend + live map	NEW	Operator dashboard; map UI; streaming chat panel

Build sequencing note: the final product is **both dispatch & navigation, fully live**. But you build **dispatch-first** and **snapshot-first**, then layer navigation and live movement on top. You lose nothing in the final product and dramatically raise the odds of finishing.

The build, phase by phase

Each phase produces something working before the next begins. Phase 0 is already done. The fine-tuning extension (★) is the one place you train a model — do it before the agents.

PHASE 0 · DONE

InferEdge — local inference engine

Llama-3.1-8B on Apple Silicon, OpenAI-compatible streaming API

Mission Already built and tested. Serves every agent in Nexa Pulse from one endpoint.

TASKS

- FastAPI server wrapping mlx-lm with a /v1/chat/completions endpoint.
- Token-by-token SSE streaming; context-window budgeting via real token counts.
- Per-request temperature / top_p through make_sampler; /health endpoint.
- Documented the MLX Metal-stream threading constraint (strong interview story).

WHAT YOU MASTER

SSE streaming

Context budgeting

MLX inference

FastAPI async

PHASE 1

Spatial + vector database foundation

PostgreSQL + PostGIS + pgvector in Docker

Mission Build the memory layer that understands both geography and meaning.

TASKS

- Dockerize PostgreSQL; enable PostGIS and pgvector extensions.
- Schema: drivers (live points), demand_zones (polygons), trips, and a policy_docs table with a vector column.
- Spatial indexes (GIST) on geometry; IVFFlat index on embeddings.
- Seed a realistic city snapshot: driver positions, zones, and a set of mock Nexa operational policy documents.

WHAT YOU MASTER

Docker Compose

Spatial indexing

Vector indexing

Schema design

SRID 4326

PHASE 2

Hybrid spatial RAG pipeline

Chunking, embeddings, and combined spatial + vector retrieval

Mission Connect the policy documents to the live map so retrieval is location-aware.

TASKS

- Chunk policy docs with RecursiveCharacterTextSplitter (overlapping chunks).
- Embed chunks with sentence-transformers, routed to the Apple GPU via .to('mps').
- Insert vectors into pgvector; store policy geo-scope where relevant.
- Write the hybrid query: cosine similarity blended with PostGIS proximity (ST_DWithin / ST_Distance).

WHAT YOU MASTER

Chunking

Embeddings

MPS routing

Cosine similarity

Hybrid retrieval

EXTENSION ★

Fine-tune a mobility-domain embedding model

The one place in the project where you actually train a model

Mission Adapt retrieval to mobility-operations vocabulary; measure the lift quantitatively.

TASKS

- Curate (query, positive, hard-negative) triplets from the policy corpus — e.g. event-surge vs. peak-hour pricing as a hard negative.
- Train all-MiniLM-L6-v2 with MultipleNegativesRankingLoss (contrastive learning), logged to Weights & Biases.
- Evaluate baseline vs. fine-tuned on NDCG@10 and MRR over a held-out query set.
- Swap the fine-tuned model into the Phase 2 pipeline and re-embed.

WHAT YOU MASTER

Contrastive learning

Hard negatives

Training loop

NDCG@10 / MRR

Domain adaptation

Weights & Biases

Why this matters: the rest of the project uses models as black boxes. This is the only phase where you train one. "Fine-tuning improved retrieval NDCG@10 by X points on mobility-ops queries" is the single most valuable ML sentence in your portfolio.

PHASE 3

Nexa-Agent — multi-agent orchestration

LangGraph state machine: supervisor + GIS, RAG, synthesis agents

Mission The brain. Reason over live map state AND policy to answer operator questions.

TASKS

- Define AgentState with Pydantic (query, sql_result, coordinates, retrieved_docs, final_answer, next_agent).
- GIS Agent: NL → PostGIS SQL via function calling → executes against the live DB (nearest drivers, demand in a zone).
- RAG Agent: runs the hybrid retrieval over policies using the fine-tuned embedder.
- Supervisor routes between agents; Synthesis Agent writes a grounded, cited recommendation (constrained to retrieved context).

WHAT YOU MASTER

LangGraph

ReAct prompting

Function calling

Agentic routing

Pydantic state

Grounding

PHASE 4 · NEW

Real-time data layer

Simulated moving drivers + live updates to the client

Mission Bring the map to life: drivers move, demand shifts, surge reacts.

TASKS

- Simulation script: drivers move along plausible paths, emit position updates, write to PostGIS continuously.
- Demand + surge logic: zones heat up and cool down; surge multipliers respond to supply/demand imbalance.
- WebSocket channel pushing live state to the frontend (positions, zones, surge).
- Start from the Phase 1 snapshot, then enable continuous movement once the agents work against static data.

WHAT YOU MASTER

Live ingestion

Moving-object queries

WebSockets

Temporal reasoning

Simulation

PHASE 5 · NEW

React operator dashboard

Live map + streaming chat panel — the experience layer

Mission Wrap the system in an interface a non-technical operator can actually use.

TASKS

- React app with a live map (Mapbox / Leaflet): driver dots by status, demand heat zones, surge overlays.
- Chat panel wired to Nexa-Agent; answers stream token-by-token using the InferEdge SSE you already built.
- Live positions update over the WebSocket channel from Phase 4.
- Citations surfaced in the UI so every recommendation shows the policy it used.

WHAT YOU MASTER

React

Mapbox / Leaflet

WebSocket client

SSE streaming UI

Product UX

PHASE 6

Evaluation, deployment & polish

Ragas LLM-as-a-judge + containerized stack

Mission Prove it works and make it presentable. (Skip heavy Prometheus/Grafana.)

TASKS

- Build a 20–30 question eval set with ground-truth answers across dispatch + navigation.
- Score with Ragas on faithfulness (hallucination) and answer relevancy; compare baseline vs. fine-tuned embedder end-to-end.
- docker-compose for db + Nexa-Agent (InferEdge runs natively on macOS for Metal access).
- README, architecture diagram, and a short demo recording of the live dashboard.

WHAT YOU MASTER

Ragas eval

Faithfulness

Answer relevancy

Multi-container Docker

Demo & docs

The sentence it all builds toward

"I built Nexa Pulse, a real-time geospatial decision-support platform for ride-hailing operations. It runs a local LLM inference server I wrote myself, reasons over a live spatial database and a fine-tuned retrieval model that improved NDCG@10 by X points, orchestrates the answer through a LangGraph multi-agent system, and streams grounded, cited recommendations into a React dashboard with a live map — all verified end-to-end with automated Ragas faithfulness scoring."